

C 语言入门教程

Grant Lee

2025 年 12 月 16 日

目录

1 C 语言：从文本文件到可执行程序的底层探秘	5
1.1 核心示例代码	5
1.2 3. 易混淆点：栈（Stack）与立即数	7
1.3 4. 总结对照表	8
1.4 5. 为什么要进行分段？	8
1.5 最小示例 HelloWorld	8
1.6 #include 与头文件	9
1.7 main 函数（程序入口）	9
1.8 语句块与分号	10
1.9 printf 与常用占位符	10
1.10 从源代码到可执行文件：将自然语言翻译为机器语言	10
2 C 语言的数据基石——变量、常量和字面量	8
2.1 数据的原形：字面量 (literal) 的概念与初步认识	8
2.2 字面量的进制与类型区分	10
2.3 浮点数的后缀	17
2.4 前后缀小结	17
2.5 字面量底层机制揭秘（ARM）	18
2.6 特殊字面量：字符串字面量 (String Literals) 底层机制解密	20
2.7 核心疑问解答	21
2.8 字面量底层硬件原理总结：三种字面量的“物流通道”	25
2.9 4. 关键思考：指针 vs 数组	26
2.10 字符串字面量：只读段与栈拷贝 (String Literals)	26
2.11 C99 复合字面量 (Compound Literals)	27
2.12 总结：字面量存储开销对比表	27
2.13 3. 例外情况：字符串与大型常量表	27
2.14 4. 立即数大小受体系结构限制	27
2.15 5. 字面量属于编译期常量	28
2.16 6. 更严谨的总结合述	28
2.17 变量与常量的基本定义	28
3 C 语言的变量与常量	29
3.1 变量 (Variables)	29
3.2 常量 (Constants)	30
3.3 三种定义“常量”符号的方式	30
3.4 总结与对比	32

目录	3
3.5 命名规则与规范	36
3.6 作用域与生命周期	36
4 从字面量 (Literal) 到数据类型	18
4.1 何为魔法数字?	18
4.2 定义与概念	19
4.3 字面量的具体区分方式	19
4.4 浮点数的后缀	23
4.5 总结	23
5 为什么我们需要数据类型	24
5.1 计算机与数学	24
5.2 数据类型的本质: 对比特模式的解释	25
5.3 数据类型决定存储格式与运算规则	26
5.4 有限位宽带来的问题: 溢出、截断与精度损失	27
5.5 小结	31
5.6 整数的存储与表示	32
6 数据类型——基础知识入门	35
6.1 小白需掌握的一些入门知识	35
7 数据类型——整数类型 (Signed Integers)	37
7.1 有符号整数 (Signed Integers)	37
7.2 无符号整数 (Unsigned Integers)	38
8 数据类型——浮点类型 (Floating-Point Numbers)	39
8.1 基本概念与名词解释	39
8.2 指数域: 偏置 (Bias) 的本质与指数	43
8.3 尾数域: 隐含位与有效数字	44
8.4 示例 1: 0.15625 (可精确表示)	48
8.5 示例 2: -7.75 (带符号)	49
8.6 示例 3: 0.1 (无法精确表示)	50
8.7 额外总结	50
8.8 字符类型 (Character Types)	53
8.9 布尔类型 (Boolean)	53
8.10 枚举类型 (Enumerations)	53
8.11 指针类型 (Pointers)	54
8.12 复合与派生类型	54

目录	4
9 一级标题：概述	55
9.1 二级标题：背景	55

1 C 语言：从文本文件到可执行程序的底层探秘

1.1 核心示例代码

为了深入理解编译器如何将代码“分门别类”地放入不同的内存段，我们使用以下包含各种存储类型变量的典型示例代码进行分析：

代码 1：内存段示例代码

```
1 #include <stdio.h>
2
3 int goble_var1 = 10;           // 全局，已初始化
4 int goble_var2;               // 全局，未初始化
5 static int goble_static_var1 = 20; // 全局静态，已初始化
6 static int goble_static_var2;     // 全局静态，未初始化
7 const int goble_const_var = 30;   // 全局常量
8
9 void func(int i) {
10     printf ("%d\n", i);          // 包含字符串字面量
11 }
12
13 int main() {
14     static int static_var1 = 70;   // 局部静态，已初始化
15     static int static_var2;        // 局部静态，未初始化
16     const int const_var = 40;      // 局部常量
17     int var1 = 30;                 // 局部变量
18     int var2;                     // 局部变量
19
20     // ... 运算逻辑 ...
21     return 0;
22 }
```

为了深入理解上述变量为何会落入不同的内存区域，我们需要追踪这段代码经历的完整构建过程。我们称之为（build）。一个完整的 C 语言构建过程包括四个主要阶段：预处理（Preprocessing）、编译（Compilation）、汇编（Assembly）和链接（Linking）。下面我们逐步解析每个阶段的工作内容及其对变量存储位置的影响。

1.1.1 预处理 (Preprocessing)

命令： `gcc -E code.c -o code.i`

在这个阶段，预处理器（cpp）处理所有以 # 开头的指令。对于我们的示例代码：

- 头文件展开： `#include <stdio.h>` 被替换为标准输入输出库的完整声明内容。
- 注释擦除：所有的 `// ...` 注释被删除，留下的全是纯粹的代码。

此时，变量还只是文本形式的符号，尚未分配内存地址。

1.1.2 编译 (Compilation)

命令： `gcc -S code.i -o code.s`

编译器（cc1）将预处理后的 C 代码翻译成汇编语言。这是决定变量“命运”的关键时刻，编译器根据变量的生命周期（Storage Duration）、链接属性（Linkage）和读写权限（Mutability）将其放入特定的段（Segment）[cite: 58]。

- 代码逻辑： `func` 和 `main` 函数的操作被翻译成汇编指令。
- 数据标记：编译器使用汇编伪指令（Directives）标记变量所属的段。
- 局部变量处理：对于 `var1` 等局部变量，编译器不会为它们生成专门的段标签，而是生成相对于栈指针的偏移量指令。

1.1.3 3. 汇编 (Assembly)

命令： `gcc -c code.s -o code.o`

汇编器（as）将汇编代码转换为机器码，生成可重定位目标文件（Relocatable Object File）。此时，文件内部已经按照“段”进行了划分：

A. .data 段（已初始化数据段） [cite_start] 此段存放程序中已初始化且初值不为 0 的全局变量和静态变量。这些数据在程序运行前就已经确定，并存在于可执行文件中 [cite: 66, 110]。

- `globle_var1` (值: 10)
- `globle_static_var1` (值: 20)
- `static_var1` (值: 70) 注意：虽然它定义在 `main` 函数内部，但由于 `static` 关键字，它的生命周期贯穿整个程序，因此存放在数据段而非栈上。

B. .bss 段（未初始化数据段） 此段存放未初始化的全局变量和静态变量。[cite_start]
关键特性： .bss 段在磁盘上的目标文件中**不占用实际存储空间**，仅记录所需空间的大小。程序加载时，操作系统会将这块内存清零 [cite: 64, 110, 112]。

- `globle_var2` (默认为 0)
- `globle_static_var2` (默认为 0)
- `static_var2` (默认为 0)

C. .rodata 段（只读数据段） [cite_start] 此段存放程序中的只读数据。加载到内存后，操作系统通常会将该页面标记为“只读”（Read-Only），任何写入尝试都会导致段错误（Segmentation Fault） [cite: 65, 111]。

- `globle_const_var` (值: 30) – 全局常量。
- [cite_start] `"%d\n"` – 字符串字面量。虽然它在代码中看起来像是 `printf` 的参数，但在底层，它作为一个匿名数组存储在 .rodata 中，指令通过地址引用它 [cite: 65]。

D. .text 段（代码段） [cite_start] 此段存放编译后的机器指令（Machine Code）。该段通常是只读的，以防止程序意外修改自身的指令逻辑 [cite: 61, 105]。

- `func` 函数的所有逻辑指令。
- `main` 函数的所有逻辑指令。

1.1.4 4. 链接 (Linking)

命令: `gcc code.o -o code`

链接器 (`ld`) 将我们的 `code.o` 与标准库合并，生成最终的可执行文件。

- **地址重定位：**链接器为每个段分配最终的虚拟内存地址。
- **符号解析：**链接器找到 `printf` 的实现并将其地址填入调用处。
- **段合并：**将多个目标文件的同类型段（如 .text）合并。

1.2 3. 易混淆点：栈 (Stack) 与立即数

并非所有变量都会被“归档”到上述的数据段中。局部变量通常在程序运行时动态创建。

- **普通局部变量 (`var1, var2`):** [cite_start] 它们存储在**栈 (Stack)** 上。在编译生成的目标文件 (.o) 中，找不到它们的固定地址，它们只存在于相对于栈指针 (SP) 的偏移量指令中 [cite: 70]。

- **局部常量** (`const_var = 40`): **陷阱**: 尽管它被声明为 `const`, 但因为它在函数内部, 编译器通常将其视为普通的栈变量 (只是编译器在编译期检查不让你修改它)。在底层优化中, 如果数值较小 (如 40), 编译器甚至可能直接将其作为**立即数** (**Immediate Value**) 嵌入到 `.text` 段的指令操作数中, 根本不会在内存中分配空间。

1.3 4. 总结对照表

变量类型	示例变量	存放位置	特点
全局/静态 (已初始化)	<code>globle_var1, static_var1</code>	<code>.data</code>	读写, 占用磁盘空间
全局/静态 (未初始化)	<code>globle_var2, static_var2</code>	<code>.bss</code>	读写, 不占磁盘空间
全局常量	<code>globle_const_var</code>	<code>.rodata</code>	只读
字符串字面量	<code>"%d\n"</code>	<code>.rodata</code>	只读, 匿名存储
局部变量	<code>var1, var2</code>	<code>Stack</code>	运行时分配, 临时
局部常量	<code>const_var</code>	<code>Stack/Text</code>	视优化情况而定
函数代码	<code>func, main</code>	<code>.text</code>	只读, 可执行

表 1: C 语言变量与内存段映射关系总结

1.4 5. 为什么要进行分段?

[cite_start] 根据《编译过程详解》, 分段不仅仅是为了分类, 更是为了系统效率和安全 [cite: 104]:

1. [cite_start] **权限管理**: 数据段可读写, 代码段和只读数据段只读。这利用虚拟内存机制防止了指令被恶意或无意篡改 [cite: 105]。
2. [cite_start] **缓存效率**: 现代 CPU 的 L1 缓存通常分为指令缓存 (I-Cache) 和数据缓存 (D-Cache)。分离存放有助于提高缓存命中率 [cite: 106]。
3. [cite_start] **内存节省**: 当多个进程运行同一个程序时, `.text` 段和 `.rodata` 段在物理内存中只需要保留一份 (共享内存), 只有 `.data` 和 `.bss` 需要为每个进程单独复制副本 [cite: 107]。

1.5 最小示例 HelloWorld

代码 2: 最小 C 程序示例

```
1
2 #include <stdio.h>           // 1) 预处理指令: 包含头文件
```



```
3
4 int main(void) {           // 2) 主函数：程序从这里开始执行
5     printf("Hello, World!\n"); // 3) 调用库函数进行输出
6     return 0;              // 4) 返回状态码 0：正常结束
7 }
```

1.6 #include 与头文件

- **#include** : #include 是预处理指令，在编译开始前，编译器会把被包含的文件内容“复制粘贴”进来。
- **stdio.h** : stdio.h 中定义了常用的输入输出函数（如 printf, scanf）。如果事先不在预处理阶段声明定义，后续却在代码中使用了 stdio.h 中的函数，那么编译器就会报错。
- 尖括号 <...> 在系统路径查找；引号 "my.h" 先在当前目录再到系统路径查找。

1.7 main 函数（程序入口）

代码 3: 常见 main 函数入口示例

```
1 int main(void)
2 // 或
3 int main(int argc, char *argv[])
```

在 C 语言中，main() 是程序的唯一入口函数，程序的执行从这里开始。

- **接受**：main 函数的参数只能是 void 或命令行参数。究其原因是因为 main 函数是由系统启动代码调用的，系统启动时，系统只知道程序名、命令行参数和环境变量。
- **返回值**：在 C 语言标准（C99、C11、C17、C23）中明确规定，main 函数必须返回一个 int 类型的值，不能是 char、float、double、void 等其他类型，用来表示程序的执行状态。

① return 0: 表示正常结束。

② return 非 0: 表示异常结束，具体值可自定义以表示不同错误类型。

1.8 语句块与分号

- 花括号 { ... } 构成语句块（复合语句），可包含声明与可执行语句。
- 大多数 C 语句以分号 ; 结尾：表达式语句、声明（含初始化）、`return` 等。

1.9 printf 与常用占位符

代码 4: printf 与占位符示例

```
1 #include <stdio.h>
2 int main(void) {
3     int a = 42; double x = 3.14159; char c = 'A';
4     printf("a=%d, x=%.2f, c=%c\n", a, x, c); // 输出 a=42, x=3.14, c=A
5     return 0;
6 }
```

1.10 从源代码到可执行文件：将自然语言翻译为机器语言

C 程序从源代码到可执行文件一般经历四个主要阶段：预处理、编译、汇编和链接。下面以 `gcc` 为例分别说明，并给出对应命令。为便于工程化，还补充多文件与库的常见用法及常见问题。

(1) 预处理 (Preprocessing) 主要工作：展开 `#include`、宏替换 (`#define`)、条件编译 (`#if/#ifdef` 等)，并去除注释，得到纯 C 源。

```
gcc -E hello.c -o hello.i
```

产物：.i 文件（预处理后的 C 代码）。

(2) 编译 (Compilation) 对 .i 做词法/语法/语义分析与优化，生成目标架构的汇编代码。

```
gcc -S hello.i -o hello.s      % 也可直接对 .c 使用 -S
```

产物：.s 文件（汇编源码）。

(3) 汇编 (Assembly) 把汇编转为可重定位的目标文件，包含代码段、数据段与符号表。

```
gcc -c hello.s -o hello.o      % 也可 gcc -c hello.c -o hello.o
```

产物：.o 文件（目标文件）。

(4) **链接 (Linking)** 将若干 .o 与所需库合并，进行符号解析与重定位，加入启动代码，生成可执行文件 (Linux 为 ELF，Windows 为 PE)。

```
gcc hello.o -o hello
```

(5) **装载与运行 (Loading & Running)** 操作系统加载可执行文件到内存，先进入运行时入口 `_start`，再调用用户的 `main()`；`main` 返回值作为进程退出码。

(6) 多文件与库的基本用法

```
gcc -c main.c -o main.o
```

```
gcc -c util.c -o util.o
```

```
gcc main.o util.o -o app % 链接生成可执行文件
```

```
ar rcs libmylib.a foo.o bar.o % 生成静态库
```

```
gcc main.o -L. -lmylib -o app2 % 使用静态库 (库放在对象文件之后)
```

```
gcc -fPIC -c foo.c -o foo.o
```

```
gcc -shared -o libmylib.so foo.o % 生成共享库
```

```
gcc main.o -L. -lmylib -o app3 % 运行时需能找到共享库
```

(7) 常见问题速查

- 头文件找不到：添加包含路径 `-Ipath`。
- 未定义引用 (`undefined reference`)：缺少对应对象文件或库，或库链接顺序在前；应将 `-lxxx` 放在使用到该库的对象文件之后。
- 库找不到：添加库路径 `-Lpath`，并确认库名 (`libxxx.a/libxxx.so` 对应 `-lxxx`)。
- 位宽/架构不匹配：确认 `-m32/-m64` 或交叉编译器是否正确。

(8) **嵌入式特例** 在无操作系统的嵌入式环境中，仍有编译与链接步骤，但会使用启动文件与链接脚本生成固件映像 (如 `.hex/.bin`)，`main()` 通常为永不返回的主循环。

构建流程：

.c 源文件 $\Rightarrow$ 预处理 (展开 `#include/#define`) $\Rightarrow$ 编译 (生成 `.o/.obj`) $\Rightarrow$ 链接 (合并库与目标文件) $\Rightarrow$ 可执行文件

命令行示例 (以 `gcc` 为例)：

```
gcc hello.c -o hello # 生成可执行文件
```

```
./hello # 运行 (Windows: hello.exe)
```